

Transactions and ACID

This note is for DBMS placement preparation. It covers transactions, ACID properties, commit, rollback, schedules, serializability, recoverability, cascading rollback, and the common theory/interview points around them.

1. Transaction

A transaction is a sequence of database operations that forms one logical unit of work.

Example:

```
BEGIN TRANSACTION;  
UPDATE Account SET balance = balance - 500 WHERE account_id = 'A';  
UPDATE Account SET balance = balance + 500 WHERE account_id = 'B';  
COMMIT;
```

This entire transfer should happen completely or not happen at all.

Common Operations in a Transaction

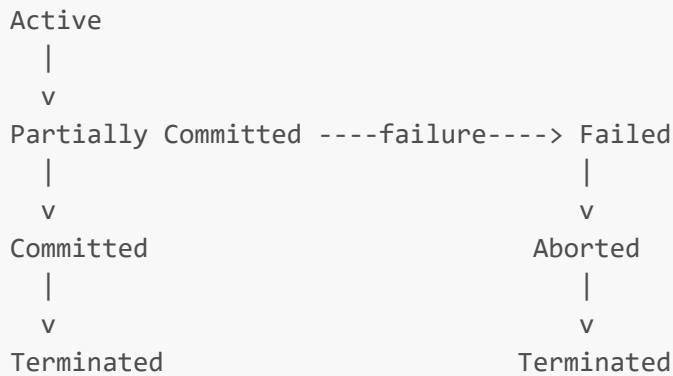
- **read_item(X)**: Reads data item **X** from the database.
- **write_item(X)**: Writes an updated value of **X** to the database.
- **commit**: Makes all changes of the transaction permanent.
- **rollback** or **abort**: Cancels the transaction and restores old values.

Transaction States

A transaction usually moves through these states:

1. **Active** The transaction is executing its read and write operations.
2. **Partially committed** The final statement has executed, but the changes may not yet be permanently stored.
3. **Committed** The transaction has completed successfully and all changes are permanent.
4. **Failed** The transaction cannot proceed because of an error, crash, deadlock, constraint violation, or explicit abort.
5. **Aborted** The transaction has been rolled back. The database is restored to the state before the transaction began.
6. **Terminated** The transaction leaves the system after commit or abort.

Transaction State Diagram



If failure occurs while active, the transaction also goes to the failed state and then to aborted.

2. Why Transactions Are Needed

Transactions are needed because databases are accessed by many users and applications at the same time. Without transaction control, the database may become inconsistent.

Problems that transactions prevent:

- Partial updates
- Lost updates
- Dirty reads
- Inconsistent reads
- Incorrect summaries
- Concurrent write conflicts
- Data corruption after system crashes

Example:

Suppose A transfers Rs. 500 to B.

```

Initial:
A = 1000
B = 2000

Step 1: A = A - 500
Step 2: B = B + 500
  
```

If the system crashes after Step 1 but before Step 2, money disappears unless transaction recovery is used.

3. ACID Properties

ACID stands for:

- Atomicity
- Consistency
- Isolation

- Durability

These are the four main properties that make transactions reliable.

4. Atomicity

Atomicity means a transaction is treated as an indivisible unit.

Either:

- All operations of the transaction are performed, or
- None of them are performed.

There is no partial transaction.

Example

```
T1:
read(A)
A = A - 500
write(A)
read(B)
B = B + 500
write(B)
```

If the transaction fails after `write(A)` but before `write(B)`, atomicity requires undoing the update to `A`.

How Atomicity Is Maintained

Atomicity is maintained using:

- Undo logs
- Write-ahead logging
- Shadow paging
- Rollback mechanisms

Placement Point

If an interviewer asks, "What happens if a transaction fails midway?", the answer is:

The system rolls back all changes made by that transaction so that the database is restored to its previous consistent state.

5. Consistency

Consistency means a transaction takes the database from one valid state to another valid state.

All integrity constraints must remain satisfied before and after the transaction.

Examples of constraints:

- Primary key constraint
- Foreign key constraint
- Unique constraint
- Check constraint
- Domain constraint
- Business rules

Example

If total balance before transfer is:

$$A + B = 1000 + 2000 = 3000$$

After transfer:

$$\begin{aligned} A &= 500 \\ B &= 2500 \\ A + B &= 3000 \end{aligned}$$

The total money remains the same, so consistency is preserved.

Important Point

Consistency is partly the responsibility of:

- The DBMS, through constraints and recovery
- The programmer, through correct transaction logic

The DBMS cannot automatically know every business rule unless it is expressed through constraints, triggers, or application logic.

6. Isolation

Isolation means concurrent transactions should not interfere with each other.

Even if many transactions execute at the same time, the final result should be as if transactions executed one after another in some serial order.

Example Without Isolation

Initial:

$$A = 1000$$

Two transactions:

```
T1: A = A - 100  
T2: A = A - 200
```

Bad interleaving:

```
T1 reads A = 1000  
T2 reads A = 1000  
T1 writes A = 900  
T2 writes A = 800
```

Correct final value should be 700, but final value becomes 800. This is a lost update problem.

How Isolation Is Maintained

Isolation is maintained using concurrency control techniques such as:

- Locks
- Two-phase locking
- Timestamp ordering
- Multiversion concurrency control
- Validation-based protocols

7. Durability

Durability means once a transaction commits, its changes are permanent, even if the system crashes immediately afterward.

Example

If a bank transaction commits and the system crashes after commit, the transferred amount must still be reflected after restart.

How Durability Is Maintained

Durability is maintained using:

- Redo logs
- Write-ahead logging
- Checkpoints
- Stable storage
- Recovery manager

Placement Point

If a transaction commits before a crash, the DBMS must redo its changes during recovery if they were not fully written to disk.

If a transaction did not commit before a crash, the DBMS must undo its changes.

8. Commit

Commit is the operation that marks a transaction as successfully completed.

After commit:

- Changes become permanent.
- Other transactions can safely observe the changes.
- The transaction cannot be rolled back normally.
- Recovery must preserve the committed changes.

SQL Example

```
BEGIN TRANSACTION;  
UPDATE Account SET balance = balance - 1000 WHERE account_id = 1;  
UPDATE Account SET balance = balance + 1000 WHERE account_id = 2;  
COMMIT;
```

Commit and Logs

Before a transaction is considered committed, its commit record is written to the log.

In write-ahead logging:

- Log records must be written before actual database pages are written.
- Commit is durable only after the commit log record reaches stable storage.

9. Rollback

Rollback cancels a transaction and undoes all changes made by it.

Rollback is used when:

- The transaction explicitly aborts.
- A constraint violation occurs.
- A deadlock victim is selected.
- A system failure occurs before commit.
- The user cancels the transaction.

SQL Example

```
BEGIN TRANSACTION;  
UPDATE Account SET balance = balance - 1000 WHERE account_id = 1;  
ROLLBACK;
```

After rollback, the balance is restored to its value before the transaction began.

Rollback vs Commit

Feature	Commit	Rollback
Meaning	Makes changes permanent	Cancels changes
Transaction result	Successful	Unsuccessful
Recovery action	Redo if needed	Undo
Changes visible later	Yes	No
Can undo normally after this?	No	Already undone

10. Schedule

A schedule is the order in which operations of multiple transactions are executed.

If transactions execute concurrently, their operations may be interleaved.

Example:

```
T1: read(A), write(A)
T2: read(A), write(A)
```

A possible schedule:

```
read1(A)
read2(A)
write1(A)
write2(A)
```

Here `read1(A)` means transaction `T1` reads item `A`.

11. Types of Schedules

Serial Schedule

A serial schedule executes one transaction completely before starting another.

Example:

```
T1: read(A), write(A), read(B), write(B), commit
T2: read(A), write(A), commit
```

Order:

T1 completely, then T2 completely

Serial schedules are easy to reason about and usually correct, but they reduce concurrency.

Non-Serial Schedule

A non-serial schedule interleaves operations of multiple transactions.

Example:

```
read1(A)
read2(A)
write1(A)
write2(A)
commit1
commit2
```

Non-serial schedules improve performance but may cause inconsistency if not controlled.

Serializable Schedule

A serializable schedule is a non-serial schedule that produces the same final result as some serial schedule.

This is the main correctness criterion for concurrent transaction execution.

12. Conflicting Operations

Two operations conflict if all these conditions are true:

1. They belong to different transactions.
2. They access the same data item.
3. At least one operation is a write.

Conflict Types

Operation Pair	Conflict?	Reason
read1(X) and read2(X)	No	Both only read
read1(X) and write2(X)	Yes	One writes
write1(X) and read2(X)	Yes	One writes
write1(X) and write2(X)	Yes	Both write
read1(X) and write2(Y)	No	Different data items

Non-conflicting operations can be swapped without changing the result.

Conflicting operations cannot be swapped freely.

13. Serializability

Serializability checks whether a concurrent schedule is equivalent to a serial schedule.

There are mainly two types:

- Conflict serializability
- View serializability

14. Conflict Serializability

A schedule is conflict serializable if it can be transformed into a serial schedule by swapping only non-conflicting operations.

Precedence Graph Method

To check conflict serializability:

1. Create one node for each transaction.
2. Add a directed edge $T_i \rightarrow T_j$ if an operation of T_i conflicts with and appears before an operation of T_j on the same data item.
3. If the graph has no cycle, the schedule is conflict serializable.
4. If the graph has a cycle, the schedule is not conflict serializable.

Edge Rules

Add edge $T_i \rightarrow T_j$ when:

- $read_i(X)$ occurs before $write_j(X)$
- $write_i(X)$ occurs before $read_j(X)$
- $write_i(X)$ occurs before $write_j(X)$

where $i \neq j$.

Example 1: Conflict Serializable

Schedule:

```
read1(A)
write1(A)
read2(A)
write2(A)
```

Conflicts:

- $write1(A)$ before $read2(A)$ gives edge $T_1 \rightarrow T_2$
- $write1(A)$ before $write2(A)$ gives edge $T_1 \rightarrow T_2$
- $read1(A)$ before $write2(A)$ gives edge $T_1 \rightarrow T_2$

Graph:

T1 -> T2

No cycle, so the schedule is conflict serializable.

Equivalent serial order:

T1, T2

Example 2: Not Conflict Serializable

Schedule:

```
read1(A)
read2(A)
write1(A)
write2(A)
read1(B)
write2(B)
```

Conflicts on A:

- `read1(A)` before `write2(A)` gives T1 -> T2
- `read2(A)` before `write1(A)` gives T2 -> T1

The graph has a cycle:

T1 -> T2 -> T1

So the schedule is not conflict serializable.

15. View Serializability

A schedule is view serializable if it is view equivalent to some serial schedule.

Two schedules are view equivalent if:

1. For every data item, if a transaction reads the initial value in one schedule, it must read the initial value in the other schedule too.
2. If T_i reads a value of X written by T_j in one schedule, then T_i must read the value of X written by T_j in the other schedule too.
3. The transaction that performs the final write on each data item must be the same in both schedules.

View vs Conflict Serializability

Point	Conflict Serializability	View Serializability
Based on	Conflicting operations	Read-from and final-write relationships
Checking method	Precedence graph	More complex
Strictness	More restrictive	Less restrictive
Relationship	Every conflict-serializable schedule is view-serializable	Not every view-serializable schedule is conflict-serializable
Practical use	Common in DBMS algorithms	Mostly theoretical

Blind Write

A blind write occurs when a transaction writes a data item without reading it first.

Example:

```
write1(X)
write2(X)
write1(X)
```

Blind writes can make a schedule view serializable but not conflict serializable.

Placement point:

If a schedule is conflict serializable, it is definitely view serializable.

If a schedule is view serializable, it may or may not be conflict serializable.

16. Recoverability

Recoverability is about whether committed transactions need to be undone due to failure of another transaction.

A schedule is recoverable if a transaction commits only after all transactions whose data it has read have committed.

Problem Scenario

```
write1(A)
read2(A)
commit2
abort1
```

Here:

- T2 reads data written by T1.

- T2 commits before T1.
- Later T1 aborts.

This is bad because T2 has committed based on dirty data from T1. Since committed transactions should not be rolled back, the schedule is not recoverable.

Recoverable Schedule

```
write1(A)
read2(A)
commit1
commit2
```

Here, T2 reads from T1, but T2 commits only after T1 commits.

So the schedule is recoverable.

Rule for Recoverability

If Tj reads a value written by Ti, then:

```
commit(Ti) must occur before commit(Tj)
```

17. Non-Recoverable Schedule

A non-recoverable schedule allows a transaction to commit even though it has read data from another uncommitted transaction.

Example:

```
write1(X)
read2(X)
commit2
abort1
```

Problem:

- T2 commits based on data written by T1.
- T1 aborts later.
- T2 has already committed invalid data.

This violates recoverability.

In real DBMS systems, non-recoverable schedules are avoided.

18. Cascading Rollback

Cascading rollback happens when the failure of one transaction forces other dependent transactions to roll back.

Example

```
write1(A)
read2(A)
write2(B)
read3(B)
abort1
```

Explanation:

- T2 reads dirty data written by T1.
- T3 reads data written by T2.
- T1 aborts.
- Therefore T2 must abort.
- Since T3 depends on T2, T3 must also abort.

This chain reaction is called cascading rollback.

Why Cascading Rollback Is Bad

It is bad because:

- Many transactions may need to be rolled back.
- Work already done by other transactions is wasted.
- Recovery becomes complex.
- System performance decreases.
- User response time increases.

19. Cascadeless Schedule

A cascadeless schedule avoids cascading rollback.

A schedule is cascadeless if a transaction reads a data item only after the transaction that last wrote that item has committed.

Rule

If T_j wants to read data written by T_i, then:

```
commit(Ti) must occur before read_j(X)
```

Example

```
write1(A)
commit1
read2(A)
write2(B)
commit2
read3(B)
commit3
```

No transaction reads uncommitted data, so cascading rollback cannot happen.

Cascadeless vs Recoverable

Every cascadeless schedule is recoverable.

But every recoverable schedule is not necessarily cascadeless.

Example of recoverable but not cascadeless:

```
write1(A)
read2(A)
commit1
commit2
```

This is recoverable because **T2** commits after **T1**.

But it is not cascadeless because **T2** read **A** before **T1** committed.

20. Strict Schedule

A strict schedule is even stronger than a cascadeless schedule.

A schedule is strict if a transaction can neither read nor write a data item until the transaction that last wrote it has committed or aborted.

Rule

If **T_i** writes **X**, then no other transaction can read or write **X** until **T_i** commits or aborts.

Example

```
write1(A)
commit1
read2(A)
write2(A)
commit2
```

This is strict.

Importance of Strict Schedules

Strict schedules are preferred because:

- They avoid dirty reads.
- They avoid dirty writes.
- They avoid cascading rollback.
- They simplify recovery.

Most practical DBMS concurrency control protocols aim to produce strict schedules.

21. Relationship Between Schedule Classes

The relationship is:

```
Strict schedules
  subset of Cascadeless schedules
    subset of Recoverable schedules
      subset of All schedules
```

In simple form:

```
Strict => Cascadeless => Recoverable
```

But the reverse is not always true.

So:

- Every strict schedule is cascadeless.
- Every cascadeless schedule is recoverable.
- Every strict schedule is recoverable.
- A recoverable schedule may not be cascadeless.
- A cascadeless schedule may not be strict.

22. Dirty Read

A dirty read happens when a transaction reads data written by another transaction that has not yet committed.

Example:

```
write1(A)
read2(A)
abort1
```

T2 read dirty data because T1 later aborted.

Dirty reads can lead to cascading rollback.

23. Dirty Write

A dirty write happens when a transaction writes a data item that was already written by another uncommitted transaction.

Example:

```
write1(A)
write2(A)
abort1
```

Dirty writes are very dangerous because recovery becomes difficult.

Strict schedules prevent dirty writes.

24. Lost Update Problem

Lost update occurs when two transactions update the same data item, but one update overwrites the other.

Example:

```
Initial X = 100

read1(X)      // T1 reads 100
read2(X)      // T2 reads 100
X = X + 10
write1(X)     // X becomes 110
X = X + 20
write2(X)     // X becomes 120
```

Correct value should be 130, but final value is 120.

25. Unrepeatable Read

An unrepeatable read occurs when a transaction reads the same data item twice and gets different values because another transaction modified it in between.

Example:

```
read1(A)      // A = 100
write2(A)     // A = 200
commit2
read1(A)      // A = 200
```

T1 reads different values for A in the same transaction.

26. Phantom Read

A phantom read occurs when a transaction repeats a query and gets a different set of rows because another transaction inserted or deleted rows.

Example:

```
-- T1
SELECT * FROM Employee WHERE salary > 50000;

-- T2
INSERT INTO Employee VALUES (101, 'Ravi', 70000);
COMMIT;

-- T1 repeats
SELECT * FROM Employee WHERE salary > 50000;
```

The second query sees an extra row. That row is a phantom.

27. Isolation Levels

SQL isolation levels define how much one transaction is isolated from others.

Isolation Level	Dirty Read	Unrepeatable Read	Phantom Read
Read Uncommitted	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible in theory
Serializable	Prevented	Prevented	Prevented

Read Uncommitted

Lowest isolation level.

Allows dirty reads.

Read Committed

A transaction can read only committed data.

Prevents dirty reads but not unrepeatable reads or phantom reads.

Repeatable Read

If a transaction reads the same row multiple times, it gets the same value.

Prevents dirty reads and unrepeatable reads.

Serializable

Highest isolation level.

Transactions behave as if executed serially.

Prevents dirty reads, unrepeatable reads, and phantom reads.

28. Two-Phase Locking

Two-phase locking is a concurrency control protocol used to ensure conflict serializability.

It has two phases:

1. **Growing phase** The transaction may acquire locks but cannot release any lock.
2. **Shrinking phase** The transaction may release locks but cannot acquire any new lock.

Lock Types

- Shared lock, also called read lock
- Exclusive lock, also called write lock

Shared Lock

If a transaction has a shared lock on **X**, it can read **X** but cannot write **X**.

Multiple transactions can hold shared locks on the same item.

Exclusive Lock

If a transaction has an exclusive lock on **X**, it can both read and write **X**.

Only one transaction can hold an exclusive lock on a data item.

Lock Compatibility Matrix

Existing Lock	Requested Shared Lock	Requested Exclusive Lock
Shared	Compatible	Not compatible
Exclusive	Not compatible	Not compatible

Important Point

Basic two-phase locking guarantees conflict serializability, but it can cause deadlocks.

29. Strict Two-Phase Locking

Strict 2PL holds all exclusive locks until commit or abort.

It guarantees:

- Conflict serializability
- Strict schedules
- Recoverability

- No cascading rollback

Strict 2PL is widely used in real DBMS systems.

30. Deadlock

A deadlock occurs when transactions wait for each other forever.

Example:

```
T1 locks A
T2 locks B
T1 requests B and waits
T2 requests A and waits
```

Neither can proceed.

Deadlock Handling

Methods:

- Deadlock prevention
- Deadlock detection
- Deadlock avoidance
- Timeout-based rollback

Deadlock Recovery

If a deadlock is detected, the DBMS chooses one transaction as victim and rolls it back.

Victim selection may depend on:

- Amount of work done
- Number of locks held
- Transaction priority
- Cost of rollback
- Number of times already rolled back

31. Recoverability and Serializability Are Different

Serializability is about correctness of concurrent execution.

Recoverability is about safe recovery after transaction failure.

A schedule can be:

- Serializable but not recoverable
- Recoverable but not serializable
- Both serializable and recoverable
- Neither serializable nor recoverable

Example: Serializable but Not Recoverable

```
write1(A)
read2(A)
commit2
commit1
```

This can be conflict serializable depending on the full schedule, but it is not recoverable because **T2** commits before **T1**, even though **T2** read from **T1**.

Key Interview Line

Serializability ensures consistency during concurrent execution, while recoverability ensures correctness when failures and aborts occur.

32. How to Check a Schedule in Exams

When given a schedule:

1. List all transactions.
2. List all read and write operations.
3. Identify conflicts.
4. Draw the precedence graph.
5. Check for cycles.
6. If no cycle, it is conflict serializable.
7. Find equivalent serial order using topological sorting.
8. Check whether any transaction reads from an uncommitted transaction.
9. Check commit order to decide recoverability.
10. Check whether reads happen only after commits to decide cascadelessness.
11. Check whether both reads and writes wait for previous writers to commit or abort to decide strictness.

33. Quick Classification Rules

Conflict Serializable

Build precedence graph.

- No cycle: conflict serializable
- Cycle: not conflict serializable

Recoverable

If **T_j** reads from **T_i**, then:

```
commit_i before commit_j
```

Cascadeless

If T_j reads X written by T_i , then:

```
commit_i before read_j(X)
```

Strict

If T_i writes X , then no other transaction may read or write X until:

```
commit_i or abort_i
```

34. Common Placement Questions

Q1. What is a transaction?

A transaction is a sequence of database operations that acts as a single logical unit of work. It must either complete fully or have no effect.

Q2. What are ACID properties?

ACID properties are Atomicity, Consistency, Isolation, and Durability. They ensure that transactions are reliable even during failures and concurrent execution.

Q3. Difference between commit and rollback?

Commit makes transaction changes permanent. Rollback cancels the transaction and undoes all its changes.

Q4. What is a schedule?

A schedule is the chronological order of operations of one or more transactions.

Q5. What is serializability?

Serializability is the property that a concurrent schedule gives the same result as some serial execution of the same transactions.

Q6. What is conflict serializability?

A schedule is conflict serializable if it can be converted into a serial schedule by swapping non-conflicting operations.

Q7. How do you test conflict serializability?

Draw a precedence graph. If the graph has no cycle, the schedule is conflict serializable. If it has a cycle, it is not conflict serializable.

Q8. What is view serializability?

View serializability means the schedule is view equivalent to some serial schedule. It preserves initial reads, read-from relationships, and final writes.

Q9. Difference between conflict and view serializability?

Conflict serializability is easier to test and more restrictive. View serializability is more general but harder to test. Every conflict-serializable schedule is view-serializable, but not every view-serializable schedule is conflict-serializable.

Q10. What is a recoverable schedule?

A schedule is recoverable if a transaction commits only after all transactions whose changes it has read have committed.

Q11. What is a non-recoverable schedule?

A non-recoverable schedule allows a transaction to commit after reading uncommitted data from another transaction that may later abort.

Q12. What is cascading rollback?

Cascading rollback occurs when aborting one transaction forces other dependent transactions to roll back because they read its uncommitted data.

Q13. What is a cascadeless schedule?

A cascadeless schedule does not allow transactions to read uncommitted data. A transaction can read a data item only after the transaction that wrote it has committed.

Q14. What is a strict schedule?

A strict schedule does not allow another transaction to read or write a data item until the transaction that last wrote it has committed or aborted.

Q15. Which is stronger: strict, cascadeless, or recoverable?

Strict is strongest, then cascadeless, then recoverable.

```
Strict => Cascadeless => Recoverable
```

Q16. Does conflict serializability guarantee recoverability?

No. Serializability and recoverability are different concepts. A schedule can be conflict serializable but not recoverable.

Q17. What is dirty read?

A dirty read occurs when a transaction reads data written by another transaction that has not committed yet.

Q18. What is dirty write?

A dirty write occurs when a transaction writes a data item already written by another uncommitted transaction.

Q19. What is lost update?

Lost update occurs when one transaction's update is overwritten by another transaction due to uncontrolled concurrency.

Q20. What does two-phase locking guarantee?

Basic two-phase locking guarantees conflict serializability. Strict two-phase locking also guarantees strictness and avoids cascading rollback.

35. Common MCQ Facts

- ACID properties guarantee reliable transaction processing.
- Atomicity means all-or-nothing.
- Consistency means constraints remain valid.
- Isolation means concurrent transactions do not interfere.
- Durability means committed changes survive crashes.
- **COMMIT** makes changes permanent.
- **ROLLBACK** undoes changes.
- A serial schedule has no interleaving.
- A non-serial schedule has interleaving.
- A serializable schedule may be interleaved but is equivalent to a serial schedule.
- Conflict serializability is checked using a precedence graph.
- A cycle in the precedence graph means not conflict serializable.
- No cycle means conflict serializable.
- View serializability is more general than conflict serializability.
- Blind writes are important in view serializability.
- Every conflict-serializable schedule is view-serializable.
- Every view-serializable schedule is not necessarily conflict-serializable.
- Every strict schedule is cascadeless.
- Every cascadeless schedule is recoverable.
- Every strict schedule is recoverable.
- Every recoverable schedule is not cascadeless.
- Every cascadeless schedule is not strict.
- Cascading rollback happens due to dirty reads.
- Strict schedules prevent cascading rollback.
- Strict 2PL guarantees conflict serializability and strictness.
- Basic 2PL can cause deadlocks.

36. Mini Practice Problems

Problem 1

Schedule:

```
read1(X)
write1(X)
read2(X)
write2(X)
commit1
commit2
```

Conflicts:

- `read1(X)` before `write2(X)`: T1 -> T2
- `write1(X)` before `read2(X)`: T1 -> T2
- `write1(X)` before `write2(X)`: T1 -> T2

No cycle.

Answer:

- Conflict serializable: yes
- Equivalent serial order: T1, T2
- Recoverable: yes, because T2 commits after T1

Problem 2

Schedule:

```
write1(X)
read2(X)
commit2
abort1
```

Answer:

- Dirty read: yes
- Recoverable: no
- Cascading rollback possible: yes
- Problem: T2 committed before T1, even though T2 read from T1

Problem 3

Schedule:

```
write1(X)
read2(X)
commit1
commit2
```

Answer:

- Recoverable: yes
- Cascadeless: no
- Reason: T2 read from T1 before T1 committed, but committed after T1

Problem 4

Schedule:

```
write1(X)
commit1
read2(X)
write2(X)
commit2
```

Answer:

- Recoverable: yes
- Cascadeless: yes
- Strict: yes

Problem 5

Schedule:

```
read1(X)
read2(X)
write1(X)
write2(X)
```

Conflicts:

- read1(X) before write2(X): T1 -> T2
- read2(X) before write1(X): T2 -> T1
- write1(X) before write2(X): T1 -> T2

Cycle:

```
T1 -> T2 -> T1
```

Answer:

- Conflict serializable: no

37. Last-Minute Revision Table

Term	Meaning
Transaction	Logical unit of database work
Atomicity	All or nothing
Consistency	Valid state to valid state
Isolation	Concurrent transactions behave independently
Durability	Committed changes survive failure
Commit	Save permanently
Rollback	Undo transaction
Schedule	Order of transaction operations
Serial schedule	Transactions execute one by one
Non-serial schedule	Operations are interleaved
Serializable schedule	Equivalent to a serial schedule
Conflict	Same item, different transactions, at least one write
Conflict serializable	Can be converted to serial by swapping non-conflicting operations
Precedence graph	Graph used to test conflict serializability
View serializable	View equivalent to a serial schedule
Recoverable	Dependent transaction commits after source transaction
Non-recoverable	Transaction commits after reading uncommitted data
Cascading rollback	One abort causes other aborts
Cascadeless	Transactions read only committed data
Strict	Transactions cannot read or write an item until previous writer commits or aborts
Dirty read	Reading uncommitted data
Dirty write	Writing over uncommitted data
Lost update	One update overwrites another
2PL	Locking protocol for conflict serializability

38. High-Yield Final Summary

Transactions are the foundation of reliable DBMS execution. ACID properties ensure that transactions remain correct even with crashes and concurrency.

Atomicity gives all-or-nothing execution. Consistency preserves database rules. Isolation protects transactions from concurrent interference. Durability guarantees that committed changes are permanent.

Schedules describe how transaction operations are ordered. Serial schedules are simple but slow. Non-serial schedules improve performance but must be checked for correctness. Serializability ensures that an interleaved schedule behaves like a serial one.

Conflict serializability is checked using a precedence graph. If the graph has a cycle, the schedule is not conflict serializable. If it has no cycle, the schedule is conflict serializable.

Recoverability deals with failure safety. A transaction must not commit before the transactions whose data it read have committed. Cascading rollback occurs when transactions read uncommitted data and are forced to abort after the original transaction aborts.

The most important relationship to remember is:

```
Strict => Cascadeless => Recoverable
```

For placement exams and interviews, be very comfortable with:

- Drawing precedence graphs
- Identifying conflicts
- Checking cycles
- Finding equivalent serial orders
- Distinguishing recoverable, cascadeless, and strict schedules
- Explaining dirty read, dirty write, lost update, and cascading rollback
- Explaining how commit and rollback relate to recovery